

به نام خدا

دانشگاه اصفهان
دانشکده مهندسی کامپیوتر

موضوع:
تحقیق سری دوم برای پروژه کارشناسی
(ابزارهای مشابه برای گزارش تحلیلی تست ها و کار با ابزار)

توسط:
محدثه آخوندی

استاد:
جناب آقای دکتر محمدرضا شهرباف

1. یک ابزاری وجود دارد به نام TestComplete:

(برای یونیت تست نیست ولی اینکه تحلیل دقیق دارد میتونه به درد بخوره)

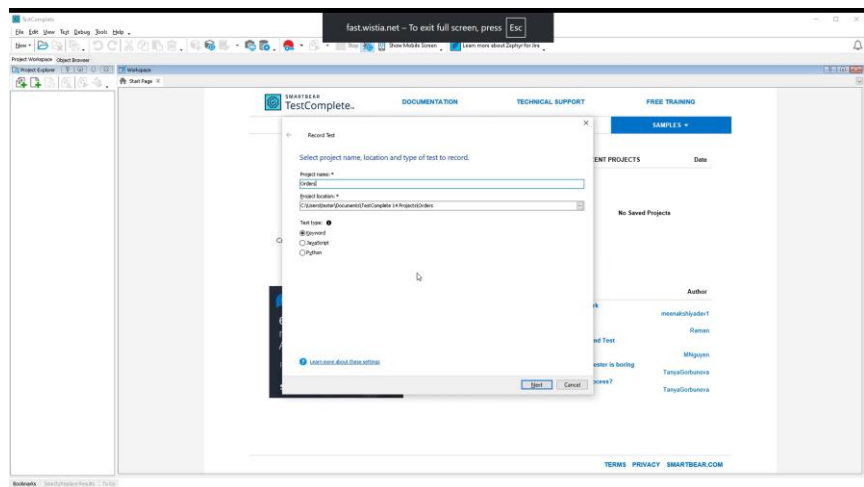
ارائه شده توسط:

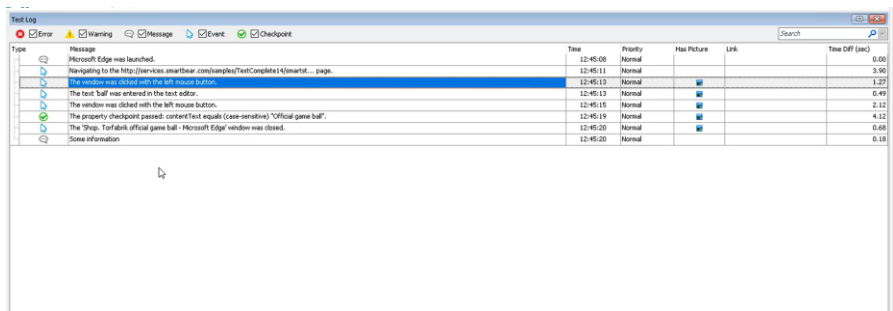
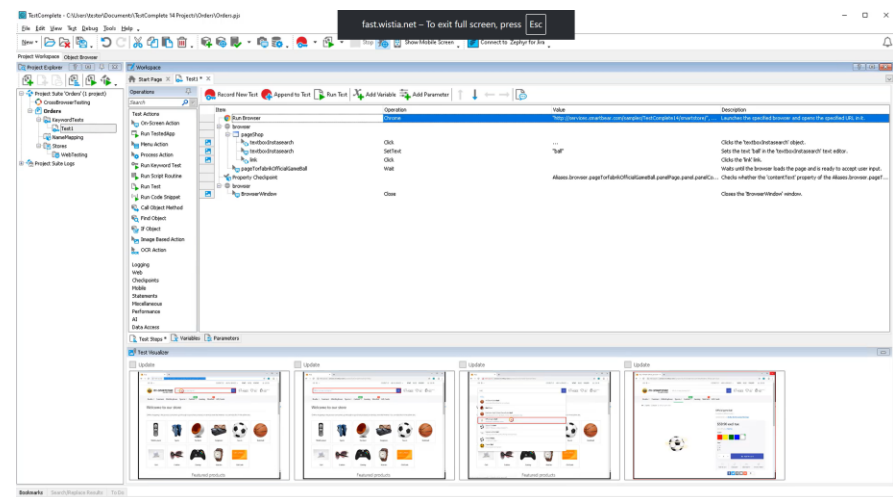
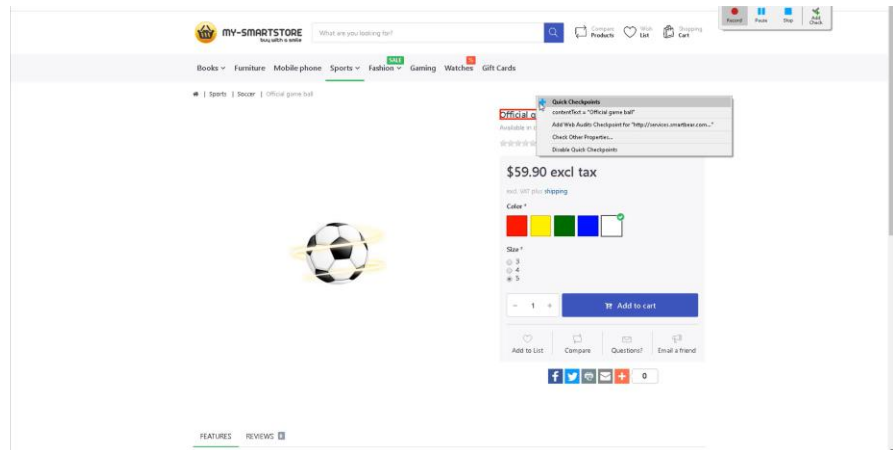
<https://smartbear.com>

TestComplete یک ابزار قدرتمند تست اتوماسیون UI است که به تیم‌های QA امکان می‌دهد تست‌های خودکار را برای برنامه‌های وب، دسکتاپ و موبایل ایجاد و اجرا کنند. این ابزار با ارائه گزارش‌های تحلیلی دقیق، علل شکست تست‌ها (مانند عدم یافتن المان‌ها یا خطاهای زمان اجرا) را شناسایی می‌کند و محل دقیق خطا را در کد تست نشان می‌دهد. TestComplete با قابلیت‌هایی مانند ضبط خودکار تست‌ها، پشتیبانی از چندین زبان برنامه‌نویسی، تشخیص هوشمند المان‌های UI و یکپارچه‌سازی با ابزارهای CI/CD، به تیم‌ها کمک می‌کند تا مشکلات را سریع‌تر تشخیص داده و با استفاده از گزارش‌های تصویری و پیشنهادات هوشمند، راه‌حل‌های مناسبی برای رفع خطاها ارائه دهند. این ابزار با نمایش تفاوت‌ها بین اجراهای موفق و ناموفق تست‌ها، امکان تحلیل عمیق‌تر نتایج و بهبود کیفیت نرم‌افزار را فراهم می‌سازد.

کاربرد: TestComplete در اصل برای «تست رابط کاربر» طراحی شده و بیشترین ارزش آن در شبیه‌سازی تعامل‌های کاربر، تشخیص المان‌های UI و تحلیل خطاهای سطح رابط است.

تصاویر کار با ابزار:



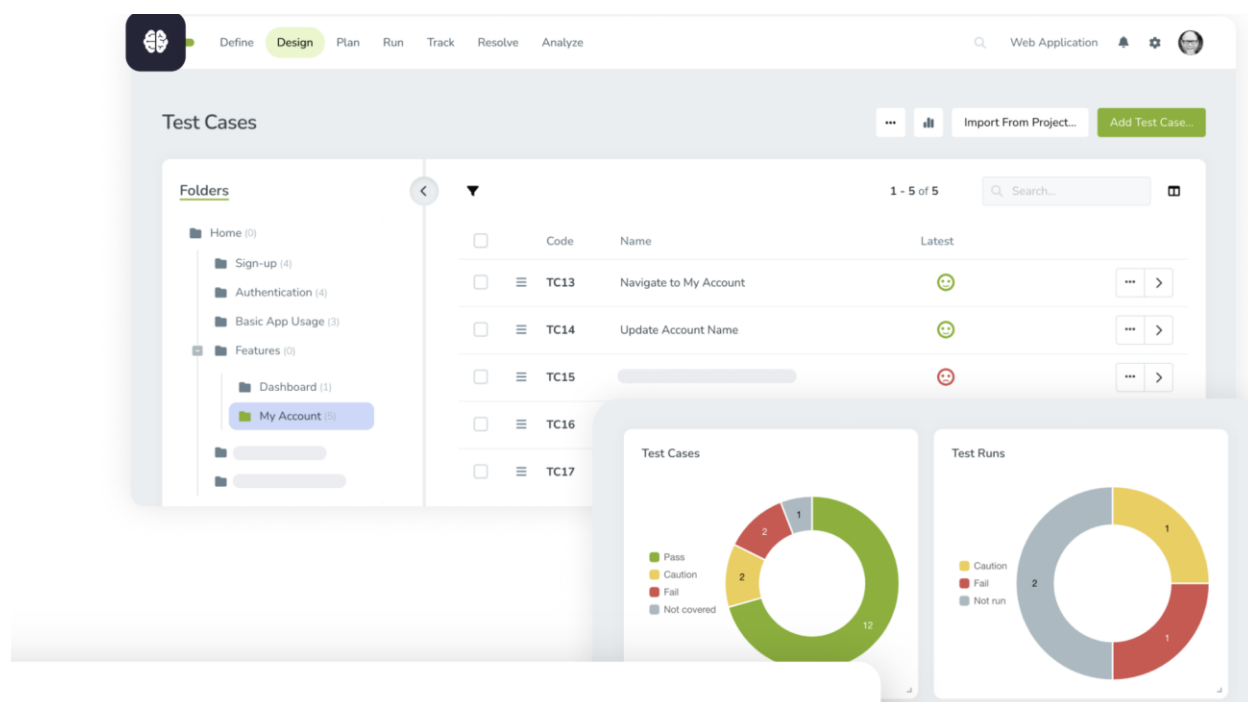


TestRail 2

عملیات فرایند تست رو ترک میکنه و میگه چقدر پوشش دهی داره تست و کجا فیل شد و کجا درست پیش رفت

گزارشهای لحظه‌ای و کاملاً سفارشی (داشبورد پیشرفت، مایلیستون، پوشش تست) در قالب وب، PDF یا ایمیل ارائه می‌دهد. نتیجه تست‌های دستی و خودکار را یکجا نشان می‌دهد و با API می‌توان خروجی یونیت-تست یا تست‌های ادغامی CI را هم وارد کرد.

بیشترین ارزش آن برای تست‌های سیستمی/رگرشنی و ردگیری یکپارچه در پروژه‌های بزرگ است؛ اما به دلیل پشتیبانی از-JUnit XML، برای یونیت-تست هم قابل استفاده است



Zebrunner 3

به کارمون نمیداد این

داشبورد ابری با ویدئوی اجرای تست، ویجت‌های نموداری و چارت‌های تعاملی ارائه می‌کند؛ خطاها را همراه اسکرین‌شات، لاگ و متادیتای محیطی تحلیل می‌کند. به‌خاطر یکپارچگی عمیق با Selenium، Cypress و Appium، عمدتاً برای تست‌های سیستمی/رگرشنی UI و موبایل کاربرد دارد؛ یونیت-تست خام را پشتیبانی نمی‌کند.

ReportPortal 4

<https://reportportal.io>

موتور ML آن شکست‌ها را خوشه‌بندی و «علّت احتمالی» را پیش‌بینی می‌کند، در کنار داشبورد Release-Health و Quality Gate خودکار. چون هر فریم‌ورکی که XML یا JSON نتیجه تولید کند را می‌پذیرد، هم یونیت-تست مانند JUnit و pytest، هم API/Integration و هم رگرشنی UI را پوشش می‌دهد؛ مناسب تیم‌های CI/CD پر حجم که به ریشه‌یابی سریع نیاز دارند.

پس از اجرای تست‌ها توسط فریم‌ورک‌های مختلف، نتایج تست به ReportPortal ارسال می‌شود. سپس این پلتفرم نتایج را تجزیه و تحلیل کرده و گزارش‌هایی دقیق و عمیق از وضعیت تست‌ها تولید می‌کند. این گزارش‌ها شامل جزئیات خطا، زمان اجرای تست‌ها،

روندهای موفقیت و شکست، و نمودارهایی برای شبیه‌سازی روندهای پیشرفت تست‌ها هستند. این ابزار به‌ویژه از قابلیت‌های AI و تحلیل داده برای شناسایی الگوهای خاص و خطاهای تکراری استفاده می‌کند

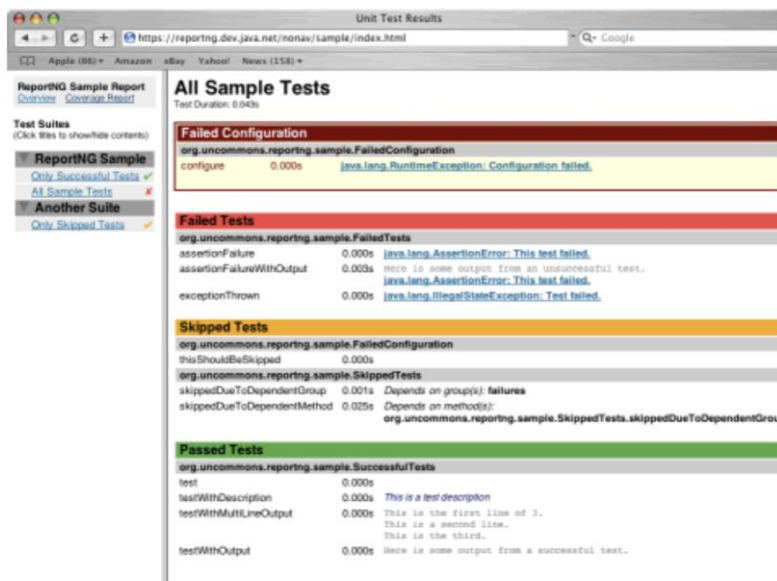
ReportNG 5

<https://testng.org/reportng>

یک پلاگین ساده XHTML برای TestNG است که خروجی پیش‌فرض را غنی‌تر می‌کند: نمودار درصد پاس/فیل، استک‌تریس خطا و پوشش را نشان می‌دهد. کاملاً در قلمروی یونیت-تست و رگرشن سبک در پروژه‌های جاوا قرار می‌گیرد؛ گزارش‌های آن بیشتر برای توسعه‌دهندگان خط مقدم مفید است تا مدیران پروژه.

این ابزار از نتایج تست‌ها که توسط چارچوب‌هایی مانند JUnit یا TestNG تولید می‌شوند، ورودی می‌گیرد. زمانی که تست‌ها در این چارچوب‌ها اجرا می‌شوند، اطلاعاتی مانند نتیجه هر تست، زمان اجرا و اطلاعات جزئی‌تر در مورد تست‌ها به ReportNG ارسال می‌شود.

این ابزار گزارشی خوانا و ساده در قالب HTML تولید می‌کند. این گزارش‌ها شامل نتایج تست‌ها، زمان اجرا، و وضعیت تست‌ها هستند که می‌تواند برای تیم‌های توسعه مفید باشد. این ابزار بیشتر برای پروژه‌هایی که نیاز به گزارش‌گیری سریع و ساده دارند مناسب است.



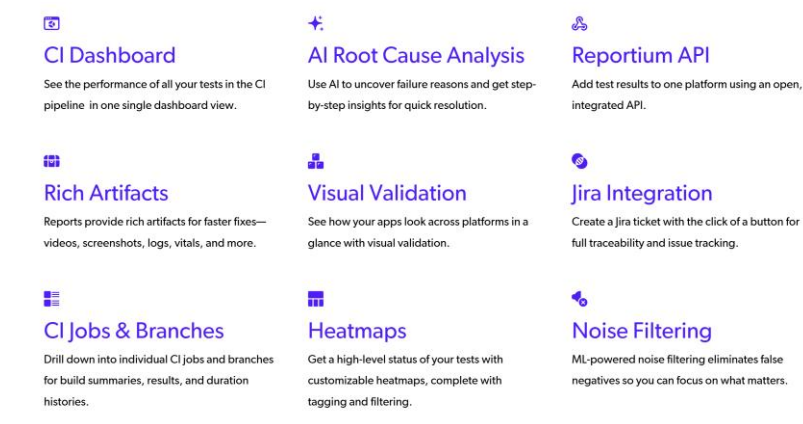
6. Perfecto Analytics & Reporting

به کارمون نمیداد

<https://www.perfecto.io/platform/analytics-reporting>

لایه تحلیلی هوشمندِ سرویس تست ابری Perfecto است که پس از هر اجرای خودکار، داده‌های آزمون را در لحظه جمع‌آوری و در قالب داشبوردهای تعاملی، هیت‌مپ، جدول مقایسه اجراهای موفق و ناموفق و گزارش‌های صفحه‌بندی-شده ارائه می‌دهد. این ماژول با بهره‌گیری از قابلیت «Smart Reporting» خطاهای تکراری را خوشه‌بندی، ریشه احتمالی شکست (کد، اسکریپت یا محیط) را مشخص و برای هر ایراد جزئیات مرحله، لاگ، اسکرین‌شات و حتی ویدیوی اجرای تست را کنار هم نمایش می‌دهد؛ افزون بر آن، شاخص‌هایی مثل روند پاس/فیل، زمان اجرا، نرخ فلیکی و پوشش دستگاه/مرورگر را به‌صورت Heatmap در اختیار تیم قرار می‌دهد. Perfecto به‌طور بومی با فریم‌ورک‌های Selenium، Cypress، Appium، XCTest و Espresso و نیز با ابزارهای CI/CD Jenkins، Azure DevOps، GitHub Actions و ... یکپارچه است؛ بنابراین گزارش‌های تحلیلی آن عمدتاً برای تست‌های رابط کاربر وب و موبایل (functional, regression, cross-browser/device) و سناریوهای اتومیشن-End-to-End در محیط‌های ابری یا واقعی طراحی شده‌اند، نه برای یونیت‌تست‌های سطح کد.

کارایی که انجام می‌دهد رو میتونیم برای یونیت تست ها استفاده کنیم و ازش ایده بگیریم:



Allure TestOps.7

یک ابزار مدیریت تست و تحلیل است که به تیم‌های توسعه کمک می‌کند تا تست‌های خود را مدیریت کنند، نتایج تست را تجزیه و تحلیل کنند و فرآیندهای تست را در سطح پروژه بهبود دهند.

Allure TestOps خودش به‌طور مستقیم برای انجام تست‌ها نیست، بلکه یک ابزار برای مدیریت و گزارش‌دهی از نتایج تست‌ها است. این ابزار به‌طور معمول با انواع تست‌ها مانند Unit Testing، Integration Testing، Functional Testing و سایر انواع تست‌ها ترکیب می‌شود. به تیم‌ها این امکان را می‌دهد که نتایج تست‌های انجام‌شده با ابزارهای مختلف مانند JUnit، TestNG، Selenium و دیگر ابزارهای تست را تجزیه و تحلیل کنند. پس می‌توان گفت این ابزار به‌طور مستقیم برای انجام تست‌ها طراحی نشده، بلکه بیشتر برای مدیریت و گزارش‌دهی نتایج تست‌ها کاربرد دارد.

نتایج تحلیلی بسیار دقیقی را از تست‌های انجام‌شده ارائه می‌دهد. این ابزار گزارش‌های دقیق و مفصل از تست‌ها، از جمله زمان اجرا، وضعیت تست‌ها (قبول/رد)، تعداد تست‌های موفق و ناموفق، و جزئیات دیگر مانند نمودارهای روند و تحلیل‌های آماری به شما

می‌دهد. این گزارش‌ها می‌توانند به تیم‌ها در شناسایی مشکلات، پیگیری روند پیشرفت تست‌ها و بهبود فرآیندهای تست کمک کنند. علاوه بر این، قابلیت‌هایی برای ایجاد داشبوردهایی به‌منظور نمایش اطلاعات تست‌ها در سطح بالاتر فراهم می‌کند که می‌تواند به مدیران پروژه یا تیم‌های توسعه کمک کند تا تصمیمات بهتری در مورد تست‌ها و رفع مشکلات اتخاذ کنند.

نمودارهای روند و تحلیل‌های آماری به‌طور مستقیم محل خطا در تست‌ها را نشان نمی‌دهند، اما می‌توانند کمک کنند تا الگوهای خاصی از خطاها شناسایی شوند. برای مثال:

نمودارهای وضعیت تست‌ها: این نمودارها ممکن است به شما نشان دهند که در کدام قسمت‌ها از تست‌ها (مانند ماژول‌های خاص یا سناریوهای مشخص) بیشترین خطاها رخ داده است. این اطلاعات می‌تواند به تیم توسعه کمک کند تا بیشتر بر روی این قسمت‌ها تمرکز کنند.

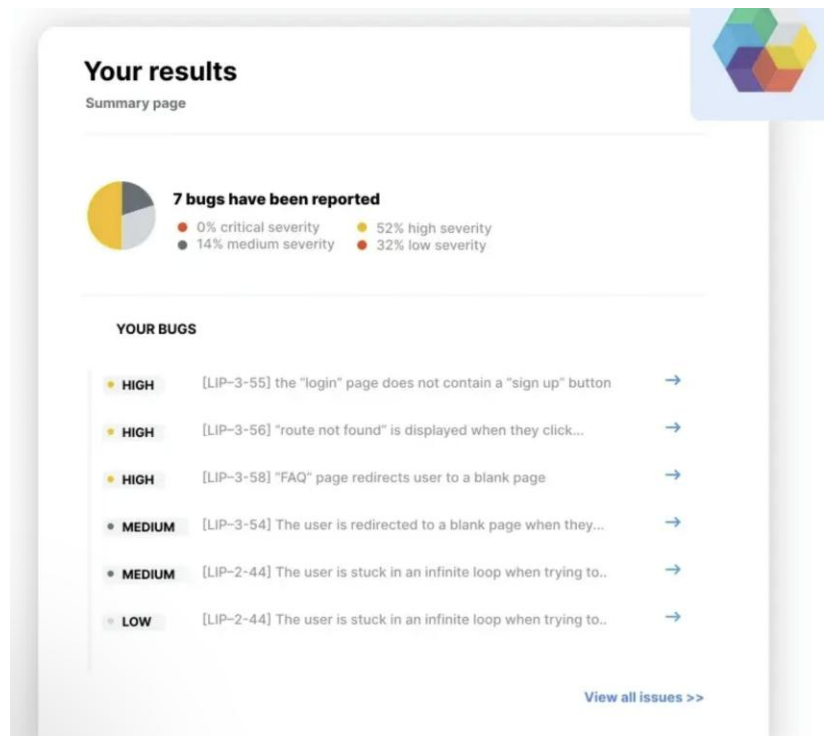
گزارش‌های دقیق تست‌ها: گزارش‌هایی که همراه با نمودارهای روند ارائه می‌شود، معمولاً شامل جزئیات بیشتری از تست‌های انجام‌شده هستند، مانند پیام‌های خطا و stack trace ها. این گزارش‌ها می‌توانند به‌طور دقیق‌تر محل خطاها و علت آن‌ها را مشخص کنند.

Global App Testing.8

به کار ما نماید چونکه برای انجام تسته و تازه برای تست سطح کد نیس

یک ابزار برای مدیریت تست‌های جامع است، اما خود ابزار به‌طور مستقیم برای انجام تست‌ها مانند Unit Testing یا Integration Testing طراحی نشده است. این ابزار بیشتر به‌عنوان یک پلتفرم تست مدیریت شده برای انجام انواع مختلف تست‌ها از جمله Functional Testing، User Acceptance Testing (UAT)، Localization Testing و Exploratory Testing استفاده می‌شود. بنابراین، این ابزار بیشتر در سطح مدیریت و هماهنگی تست‌ها در فرآیند توسعه نرم‌افزار کاربرد دارد و به‌طور مستقیم مرتبط با Unit Testing یا Integration Testing نیست.

نتایج تحلیلی دقیق و عمیق در مورد تست‌های انجام‌شده ارائه می‌دهد. این ابزار گزارش‌هایی کامل از نتایج تست‌ها فراهم می‌کند که شامل ویدیوهای تست، گزارش‌های خطا، اسکرین‌شات‌ها و جزئیات دقیق می‌شود. علاوه بر این، این پلتفرم معمولاً نتایج را به‌صورت جامع و دقیق جمع‌بندی می‌کند تا تیم‌های توسعه بتوانند مسائل را به سرعت شناسایی کرده و حل کنند. گزارش‌ها به تیم‌ها کمک می‌کند تا درک دقیقی از مشکلات کاربران نهایی، عملکرد نرم‌افزار و تجربه کلی کاربری بدست آورند.



برای اون ابزار هایی که ابی شدن :|||||

بررسی AI

مقایسه و جمع بندی						
ابزار	تحلیل نتایج تست های دیگر	تست های واحد	تست های یکپارچگی	تست های دستی	تحلیل علت و محل خطا	پیشنهاد راه حل
Allure TestOps	بله	بله (JUnit, TestNG, pytest, Jest, Mocha)	بله (Cucumber, Selenium, Webdriver.io)	بله	پیام خطا، stack trace، لینک به نقص	خیر، اما لینک به Jira
ReportNG	بله (فقط TestNG)	بله (TestNG)	بله (TestNG)	خیر	پیام خطا، stack trace، اسکرین شات	خیر
ReportPortal	بله	بله (JUnit, TestNG, Jasmine, pytest)	بله (Cucumber, JBehave, SpecFlow, Selenium)	بله	پیام خطا، stack trace، تحلیل AI	بله (تحلیل AI)

می‌تواند نتایج تست‌هایی که توسط ابزارهای دیگر اجرا شده‌اند را تحلیل کند، اما بیشتر برای تست‌هایی که در اکوسیستم خودش اجرا می‌شوند مناسب است. این فریم‌ورک اتوماسیون مبتنی بر کلیدواژه، با کتابخانه‌های متنوع (مانند Selenium ، Appium) ادغام می‌شود و گزارش‌های HTML جامعی از نتایج تست تولید می‌کند. نتایج تست معمولاً در قالب XML جمع‌آوری شده و به گزارش‌های HTML تبدیل می‌شوند. برای تحلیل نتایج ابزارهای غیرمرتبط، نیاز به ادغام سفارشی یا استفاده از خطوط لوله CI/CD است.

عمدتاً برای تست‌های پذیرش (Acceptance Testing)، توسعه مبتنی بر تست پذیرش (ATDD) و توسعه مبتنی بر رفتار (BDD) طراحی شده است، اما با استفاده از کتابخانه‌های مناسب، از تست‌های واحد (مانند ادغام با unittest در Python و تست‌های یکپارچگی) مانند تست‌های وب با Selenium یا تست‌های API نیز پشتیبانی می‌کند. گزارش‌ها شامل آمار اجرای تست، لاگ‌های دقیق، و نمودارهای بصری (مانند وضعیت‌های موفق، شکست، یا نادیده گرفته شده) هستند.

این ابزار به‌طور خاص برای تست‌های واحد طراحی نشده، اما با کتابخانه‌های مناسب می‌تواند برای تست‌های واحد استفاده شود

Klov (ExtentReports).10

صرفاً گزارش تهیه می‌کند از تست‌های اجرا شده با ابزارهای دیگر

برای تحلیل نتایج تست‌های تولیدشده توسط فریم‌ورک‌های مختلف طراحی شده است. این ابزار به‌عنوان یک سرور گزارش‌گیری وب‌محور عمل می‌کند که نتایج را از ExtentReports جمع‌آوری و تجسم می‌کند. با فریم‌ورک‌هایی مانند JUnit، TestNG، Selenium و Cucumber ادغام می‌شود و نتایج را از طریق API آن دریافت می‌کند.

نتایج تست‌های واحد از JUnit، TestNG، تست‌های یکپارچگی از Selenium، Cucumber و تست‌های عملکردی/انتها را پشتیبانی می‌کند. همچنین می‌تواند نتایج تست‌های دستی را با ادغام با ExtentReports تحلیل کند، که آن را برای انواع مختلف تست مناسب می‌کند.

برای تست‌های واحد و تست‌های یکپارچگی مناسب است، زیرا نتایج فریم‌ورک‌هایی مانند JUnit و TestNG (برای تست‌های واحد) و Selenium یا Cucumber (برای تست‌های یکپارچگی) را پردازش می‌کند.

گزارش‌های تعاملی با تحلیل‌های تاریخی، نمودارهای بصری و لاگ‌های دقیق ارائه می‌دهد. این گزارش‌ها شامل وضعیت تست‌ها، زمان اجرا، اسکرین‌شات‌ها و پیام‌های خطا هستند که به شناسایی محل شکست‌ها (مانند مراحل خاص تست) کمک می‌کنند. با این حال، تحلیل خودکار علت شکست یا پیشنهاد راه‌حل ارائه نمی‌دهد و نیاز به بررسی دستی لاگ‌ها و stack trace دارد.

JUnit Plugin for Jenkins.11

برای فقط junit کار میکنه و خب محل خطا که مربوط به کدام مثلاً تابع هست رو میگه ولی راه حل نمیده

برای جمع‌آوری و تحلیل نتایج تست‌هایی که توسط JUnit و سایر فریم‌ورک‌های سازگار مانند TestNG با گزارش‌های XML سازگار با JUnit تولید شده‌اند، طراحی شده است. این افزونه گزارش‌های XML تولیدشده در خطوط لوله CI/CD Jenkins را پردازش می‌کند.

افزونه عمدتاً برای تست‌های واحد از طریق JUnit طراحی شده که در پروژه‌های Java رایج است. همچنین می‌تواند نتایج تست‌های یکپارچگی را تحلیل کند، اگر تست‌ها با JUnit نوشته شده یا گزارش‌های XML سازگار تولید کنند (مانند تست‌های Selenium یا TestNG). تست‌های دستی را مستقیماً پشتیبانی نمی‌کند.

این افزونه گزارش‌های گرافیکی با نمایش روند تست‌ها، تعداد تست‌های موفق/شکست‌خورده، و داده‌های تاریخی ارائه می‌دهد. اطلاعات دقیق شکست‌ها، شامل نام تست، پیام‌های خطا و stack trace، محل شکست‌ها (مانند متد یا کلاس خاص) را مشخص می‌کند. اعلان‌هایی برای شکست تست‌ها یا ناپایداری بیلد ارائه می‌دهد، اما تحلیل خودکار علت شکست یا پیشنهاد راه‌حل ندارد. گزارش‌ها برای توسعه‌دهندگان آشنا با JUnit و Jenkins قابل استفاده هستند.

SpiraTest.12

تحلیل دقیقی نداره صرفاً گزارش‌های بصری میده از تست‌های اجرا شده توسط پلتفرم‌های دیگه رو

SpiraTest یک ابزار مدیریت تست جامع است که می‌تواند نتایج تست‌های دستی و خودکار اجرا شده توسط ابزارهای دیگر را تحلیل کند. این ابزار با فریم‌ورک‌های اتوماسیون مانند JUnit، TestNG، Selenium و Ranorex ادغام می‌شود و نتایج را از طریق پلاگین‌ها یا API جمع‌آوری می‌کند.

از تست‌های واحد، تست‌های یکپارچگی، تست‌های عملکردی، تست‌های پذیرش و تست‌های دستی پشتیبانی می‌کند. این ابزار می‌تواند نتایج JUnit و TestNG برای تست‌های واحد و یکپارچگی، Selenium (برای تست‌های یکپارچگی وب) و تست‌های دستی تعریف‌شده در پلتفرم خود را تحلیل کند

داشبوردهای قابل تنظیم و گزارش‌هایی با جزئیات نتایج تست، شامل وضعیت‌های موفق/شکست‌خورده، ردیابی نقص‌ها و روندهای اجرای تست ارائه می‌دهد. شکست‌ها به تست‌ها و نیازمندی‌های خاص لینک می‌شوند و پیام‌های خطا و لاگ‌ها محل شکست را مشخص می‌کنند. با این حال، تحلیل خودکار علت شکست یا پیشنهاد راه‌حل ارائه نمی‌دهد و نیاز به بررسی دستی دارد.

برای تست های واحد نیست و تفاوتش با بقیه اینه که با AI راه حل میده برای خطا ها

یک پلتفرم تست خودکار مبتنی بر هوش مصنوعی است که عمدتاً نتایج تست های اجرا شده در محیط خودش را تحلیل می کند، اما می تواند با ابزارهای دیگر از طریق خطوط لوله (CI/CD مانند Jenkins ، GitHub Actions) ادغام شود. برای مثال، می تواند نتایج تست های Selenium را در صورت ادغام مناسب تحلیل کند.

عمدتاً برای تست های عملکردی و تست های انتها به انتها طراحی شده، به ویژه برای برنامه های وب. همچنین از تست های یکپارچگی مانند تست تعاملات UI با API های بک اند پشتیبانی می کند. برای تست های واحد کاربرد محدودی دارد، اما می تواند با فریم ورک هایی مانند Jest یا Mocha ادغام شود.

برای تست های واحد مناسب نیست، زیرا تمرکز آن بر تست های UI و عملکردی است. برای تست های یکپارچگی، به ویژه در برنامه های وب، بسیار مناسب است، زیرا تعاملات بین UI و سرویس های بک اند را تست می کند

گزارش های دقیقی با وضعیت تست ها، فعالیت های تیمی و الگوهای شکست ارائه می دهد که با تحلیل های مبتنی بر هوش مصنوعی تقویت شده اند. این ابزار شکست ها را با کدهای دلیل (Reason Codes) طبقه بندی می کند و علل احتمالی مانند تغییرات UI یا مشکلات شبکه را شناسایی می کند. اسکرین شات ها، لاگ ها و ویدئوهای اجرای تست محل شکست را مشخص می کنند. هوش مصنوعی Testim می تواند راه حل هایی برای تست های ناپایدار (مانند به روزرسانی لوکاتورها) پیشنهاد دهد، که آن را از سایر ابزارها متمایز می کند.

BrowserStack Test Reporting and Analytics.14

BrowserStack Test Reporting and Analytics برای تحلیل نتایج تست های اجرا شده توسط فریم ورک های مختلف طراحی شده است. این ابزار از طریق SDK های بومی یا آپلود گزارش های XML در قالب JUnit با فریم ورک های متعددی مانند JUnit ، TestNG ، Pytest ، Mocha ، Cucumber ، و Robot Framework ادغام می شود. این قابلیت به تیم ها امکان می دهد تا نتایج تست های اجرا شده در محیط های محلی یا ابری را در یک داشبورد متمرکز تحلیل کنند

این ابزار از تست های واحد مانند JUnit ، TestNG ، Mocha. تست های یکپارچگی مانند Selenium،Cucumber برای تست های API و وب، تست های انتها به انتها (E2E) ، و تست های API پشتیبانی می کند. همچنین می تواند گزارش های Allure را پردازش کند، که آن را برای انواع مختلف تست های خودکار مناسب می کند.

گزارش های پیشرفته ای با داشبورد متمرکز ارائه می دهد که شامل وضعیت تست ها (موفق، شکست خورده، نادیده گرفته شده)، زمان اجرا، تحلیل روندهای تاریخی، و شناسایی خودکار تست های ناپایدار (flaky tests) با استفاده از هوش مصنوعی است. این ابزار دلایل شکست ها مانند خطاهای شبکه یا تغییرات UI را با کدهای دلیل (reason codes) طبقه بندی می کند و اطلاعات زمینه ای مانند لاگ ها، اسکرین شات ها و ویدئوها را برای عیب یابی ارائه می دهد. تحلیل های AI محور آن می توانند علل ریشه ای شکست ها را شناسایی کرده و پیشنهاداتی برای رفع آن ها ارائه دهند، که این ابزار را از بسیاری ابزارهای دیگر متمایز می کند

JUnit HTML Report.15

JUnit HTML Report لبه طور خاص برای تحلیل نتایج تست های اجرا شده توسط فریم ورک JUnit طراحی شده است و مستقیماً نتایج سایر فریم ورک ها را تحلیل نمی کند. گزارش های HTML معمولاً از طریق ابزارهای اضافی مانند Maven Surefire Plugin یا افزونه های Jenkins تولید می شوند.

JUnit HTML Report لعمدتاً برای تست های واحد طراحی شده است، زیرا JUnit یک فریم ورک تست واحد برای Java است. با این حال، اگر تست های یکپارچگی با JUnit نوشته شوند (مانند تست های تعاملات API یا پایگاه داده)، گزارش های HTML می توانند نتایج آن ها را نیز نمایش دهند. این گزارش ها شامل جزئیات تست مانند نام تست، وضعیت، زمان اجرا و پیام های خطا هستند.

گزارش های HTML تولید شده توسط JUnit گزارش های ساده ای ارائه می دهند که شامل تعداد تست های اجرا شده، وضعیت (موفق، شکست خورده، نادیده گرفته شده)، زمان اجرا، پیام های خطا و stack trace هستند. این گزارش ها محل شکست ها (مانند متد یا کلاس خاص) را مشخص می کنند، اما تحلیل خودکار علل شکست یا پیشنهاد راه حل ارائه نمی دهند.

XUnit Report Viewer.16

برای تحلیل گزارش های XML تولید شده توسط فریم ورک های xUnit مانند JUnit، NUnit، و xUnit.net و سایر فریم ورک هایی که خروجی XML سازگار تولید می کنند، طراحی شده است. این ابزار گزارش های XML را به صورت خودکار جمع آوری کرده و به گزارش های HTML تعاملی با قابلیت فیلتر تبدیل می کند.

برای تست های واحد و تست های یکپارچگی مناسب است، به شرطی که نتایج در قالب XML سازگار با xUnit ارائه شوند. این ابزار بیشتر برای پروژه هایی با فریم ورک های xUnit مانند JUnit، NUnit استفاده می شود و برای تست های پذیرش یا E2E کاربرد محدودی دارد.

گزارش های HTML تعاملی تولید می کند که شامل آمار تست (موفق، شکست خورده، نادیده گرفته شده)، جزئیات تست های فردی، پیام های خطا، stack trace و قابلیت فیلتر برای جستجوی تست های خاص است. این گزارش ها محل شکست ها را مشخص می کنند، اما تحلیل خودکار علل شکست یا پیشنهاد راه حل ارائه نمی دهند.

Lighthouse.17

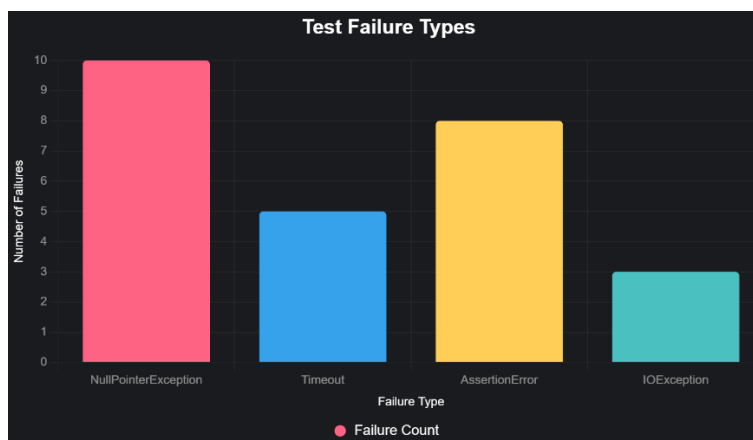
به کار نمیداد

Lighthouse برای تحلیل نتایج تست های اجرا شده توسط ابزارهای دیگر طراحی نشده است. این ابزار یک ابزار ممیزی خودکار برای ارزیابی عملکرد، دسترسی پذیری (Accessibility)، بهینه سازی موتور جستجو (SEO) و بهترین شیوه های وب است. Lighthouse گزارش هایی تولید می کند که مستقیماً از ممیزی های وب آن به دست می آیند، نه از نتایج تست های واحد یا یکپارچگی دیگر فریم ورک ها.

برای تست‌های عملکرد و کیفیت وب طراحی شده و بر ممیزی صفحات وب تمرکز دارد. در سناریوهای تست یکپارچگی، ممکن است به‌صورت غیرمستقیم برای ارزیابی عملکرد UI استفاده شود، اما کاربرد اصلی آن در تست‌های عملکرد و دسترسی‌پذیری است.

مرحله دوم

یه عکسی پیدا کردم برای نمودار تحلیلی تست‌ها: یه چیز کاملاً رندوم 😊



دوتا ابزار برخورد کردیم باهاش یکی Allure و دیگری Allure TestOps

Allure: یک ابزار گزارش‌گیری متن‌باز (open-source) برای تولید گزارش‌های زیبا و قابل خواندن از نتایج تست خودکار.

نمایش نتایج تست‌های خودکار (JUnit, TestNG, Pytest, NUnit) به شکل HTML با نمودارها و جزئیات دقیق هر تست.

Allure TestOps: یک پلتفرم مدیریت کیفیت و تست جامع (TestOps platform) که توسط همان تیم Allure توسعه داده شده، اما محصول تجاری (Enterprise) است.

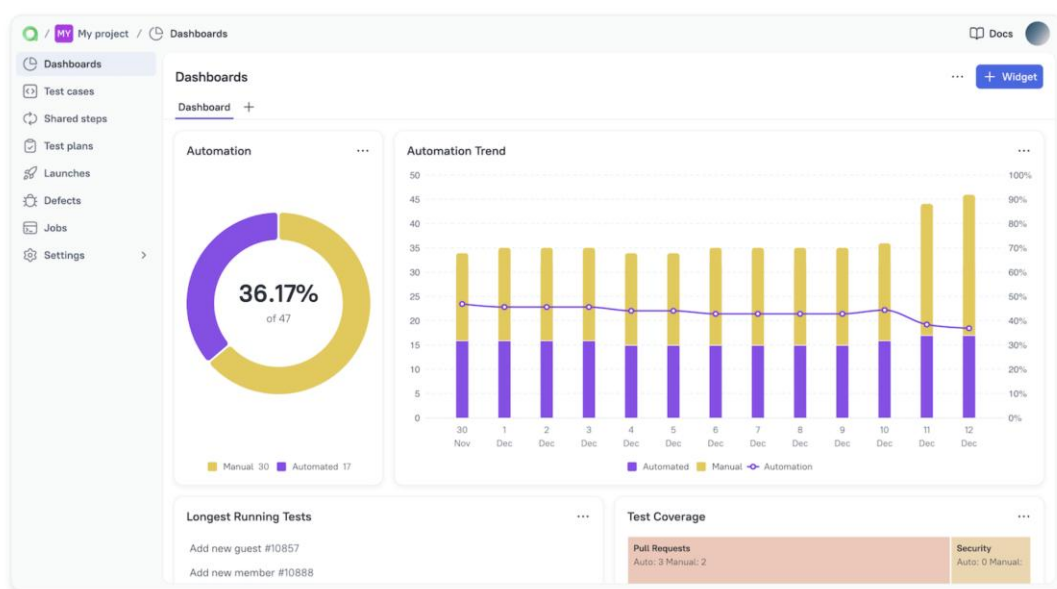
ویژگی	Allure (allure2)	Allure TestOps
نوع	ابزار گزارش‌گیری متن‌باز	پلتفرم مدیریت کیفیت و تست تجاری
نصب	لوکال (CLI)	سرور یا Cloud
هزینه	رایگان	تجاری
مدیریت تست کیس	ندارد	دارد
تست دستی	ندارد	دارد
گزارش‌گیری از نتایج	دارد	دارد با تحلیل پیشرفته
CI/CD Integration	دارد	دارد با امکانات بیشتر
تحلیل روند کیفیت	ندارد	دارد
همکاری تیمی	ندارد	دارد

من با ابزار Allure کار کردم به دلایل زیر :

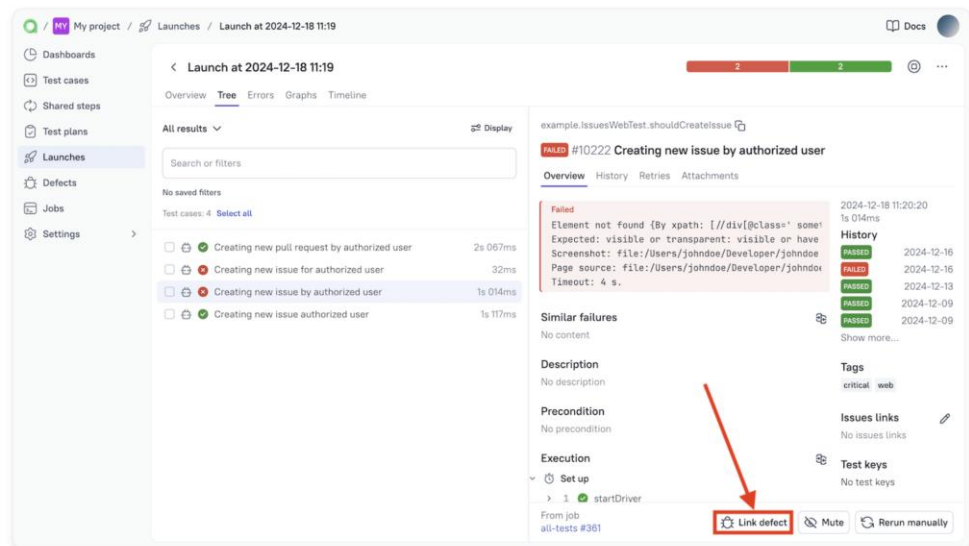
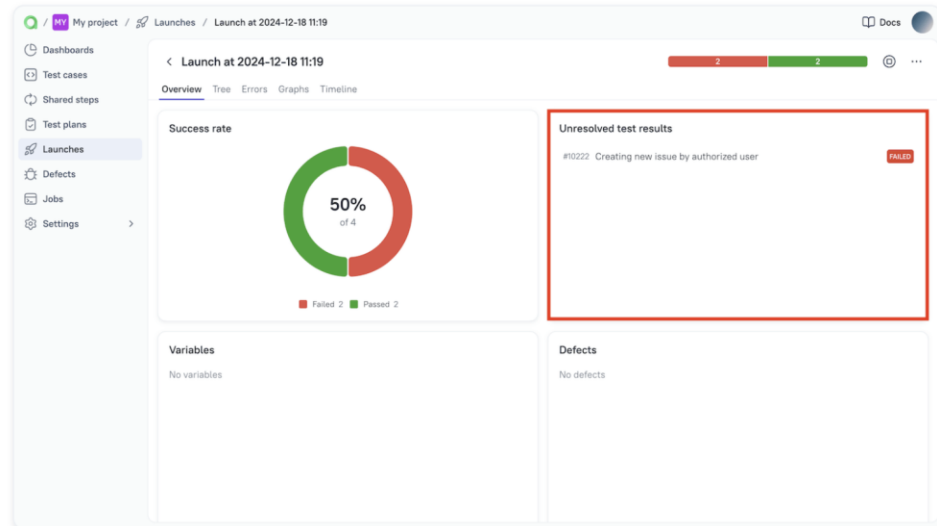
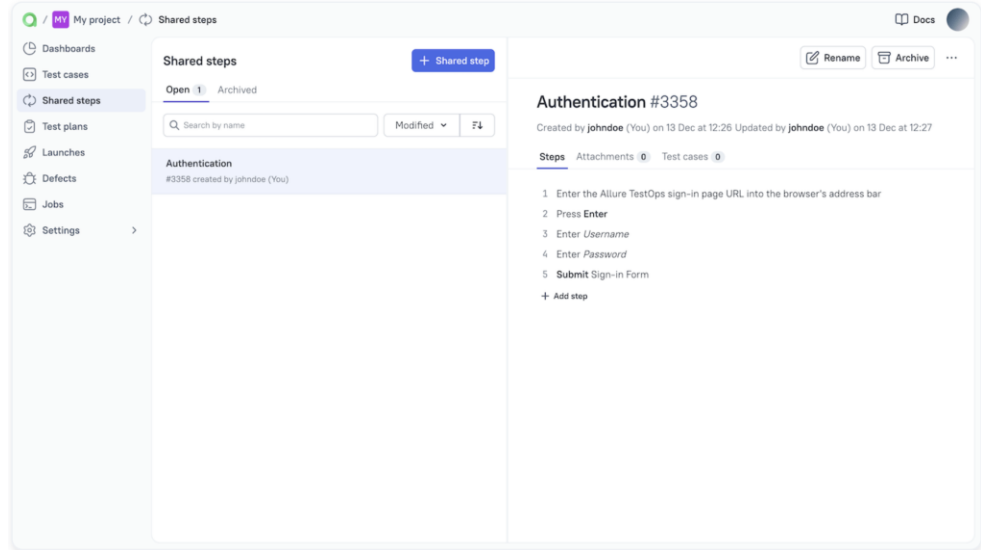
1. اگر فقط نیاز به گزارش نتایج تست خودکار داری.
2. اگر تیم کوچک هستی یا می‌خواهید هزینه‌ای نداشته باشید.
3. اگر گزارش ساده کافی است.

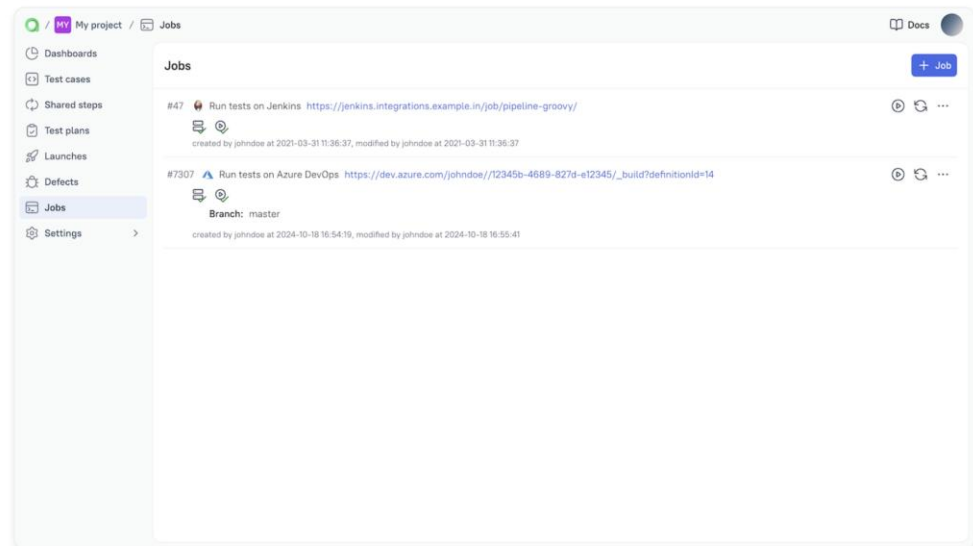
و ابزار Allure TestOps برای ایجاد یک مرکز واحد برای کیفیت پروژه کاربرد دارد.

1. تصاویر زیر مربوط به ابزار Allure TestOps است:



The screenshot shows the 'Test cases' page in Allure TestOps. The left sidebar is identical to the dashboard view. The main area is divided into two columns. The left column, titled 'All test cases', contains a search bar and a list of 20 test cases. The first test case, 'Authenticate with username and password', is highlighted with a red box. The right column displays the details for the selected test case, which is marked as 'ACTIVE #483420'. It includes tabs for Overview, History, Attachments, Mutes, Defects, and Change Log. The 'Overview' tab is active, showing the test case's description, preconditions (e.g., 'The web browser is installed and functioning correctly'), a scenario with six steps (e.g., 'Open Web Browser', 'Navigate to Sign-in Form'), and an expected result. On the right side of the details view, there are sections for 'Links', 'History' (showing a 'PASSED' status on 2024-12-12), 'Tags', 'Issues links', 'Test keys', 'Members', 'Custom Fields', and 'Mutes'.





2. گزارش کار با ابزار Allure :

مقدمه

با هدف آشنایی و کار با ابزار Allure برای گزارش‌دهی تست‌های خودکار، پروژه‌ای را آغاز کردم. با توجه به استفاده از سیستم‌عامل ویندوز 11 و وجود پیش‌نیازهایی نظیر پایتون، کتابخانه pip، و IDE های VS Code و PyCharm، مراحل نصب، تنظیم، و تست ابزار Allure را به‌صورت مرحله به مرحله طی کردم. هدف اصلی، یادگیری فیچرهای Allure و امکان توسعه و مشارکت در پروژه متن‌باز آن بود.

نصب و راه‌اندازی پیش‌نیازها

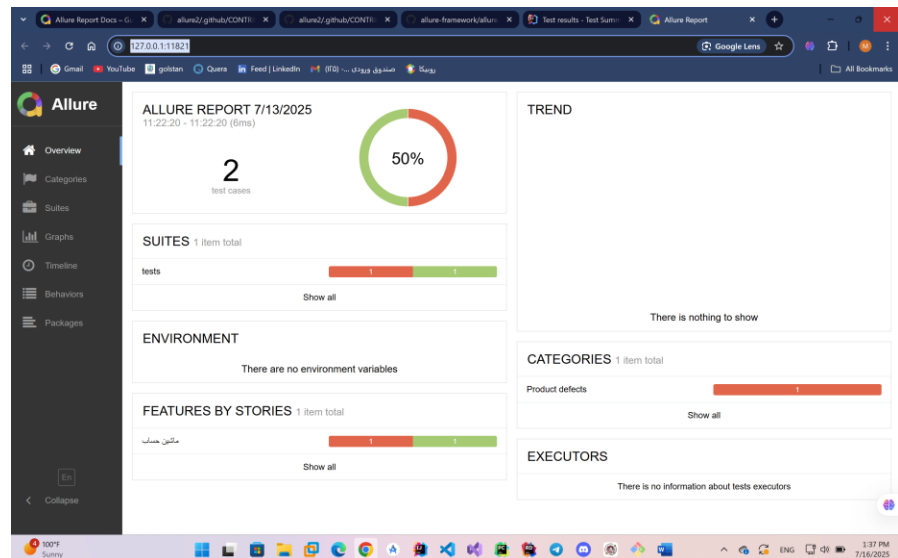
ابتدا وضعیت سیستم را بررسی کردم. پایتون 3.12.2 و pip 25.0.1 نصب‌شده بودند، اما Java که برای Allure ضروری است، شناسایی نشد. بنابراین، آخرین نسخه (JDK (Java Development Kit را از وب‌سایت رسمی Oracle دانلود و نصب کردم. پس از نصب، با دستور `java -version` در Command Prompt تأیید کردم که Java فعال است. سپس محیط VS Code را به‌روزرسانی کردم تا با نسخه‌های جدید سازگار شود.

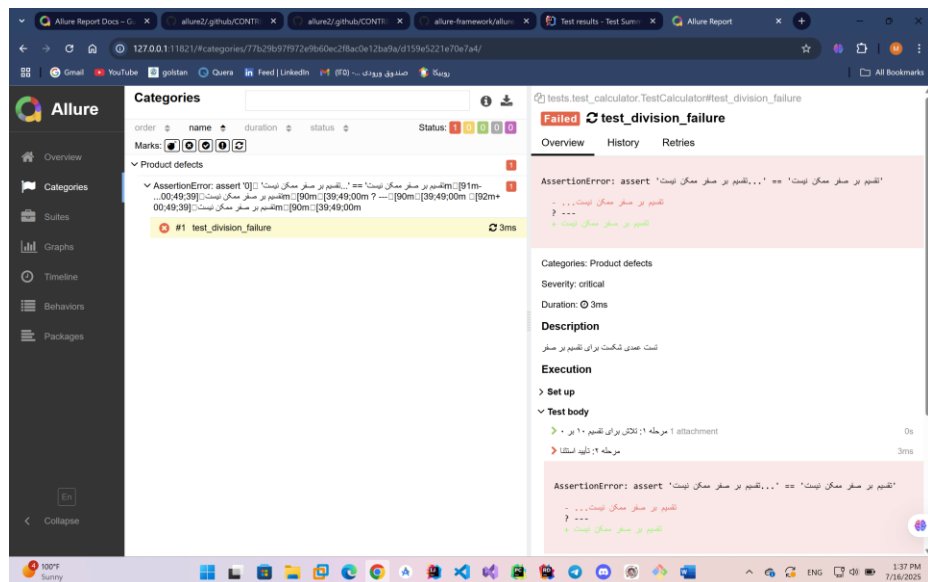
نصب و پیکربندی Allure

برای نصب Allure ، ابتدا از طریق pip کتابخانه allure-pytest را با دستور `pip install allure-pytest` نصب کردم. سپس نسخه مستقل Allure CLI را از سایت رسمی allure.qameta.io دانلود و فایل ZIP را در مسیر دلخواه استخراج کردم. مسیر `bin` را به متغیر محیطی `PATH` ویندوز اضافه کردم و با دستور `allure --version` در Command Prompt نصب را تأیید کردم.

نوشتن و اجرای تست‌های پایتون با Pytest

با استفاده از vs code ، یک پروژه پایتون جدید ساختم و یک فایل تست با نام `test_calculator.py` ایجاد کردم. در این فایل، از دکوراتورهای Allure مثل `@allure.feature` و `@allure.description` و مابقی دکوراتورها برای تعریف فیچرها و سناریوها استفاده کردم. تست‌ها را با Pytest اجرا کردم و با دستور `pytest tests/ --alluredir=allure-results` allure-results را پوشه در پوشه تولید کردم. نهایتاً با دستور `allure generate allure-results --clean -o allure-report` allure گزارش html ان را ایجاد کردم و با `allure open allure-report` ان را در مرورگر مشاهده کردم.





کد calculator.py

```
class Calculator:
    def add(self, a, b):
        """جمع دو عدد"""
        return a + b

    def divide(self, a, b):
        """تقسیم دو عدد"""
        if b == 0:
            raise ValueError("تقسیم بر صفر ممکن نیست")
        return a / b
```

کد test_calculator.py

```
import pytest
from allure_commons.types import AttachmentType
import allure
from src.calculator import Calculator

class TestCalculator:
    @pytest.fixture
    def calc(self):
        return Calculator()

    @allure.feature("ماشین حساب")
    @allure.severity(allure.severity_level.NORMAL)
    @allure.description("تست جمع دو عدد")
```

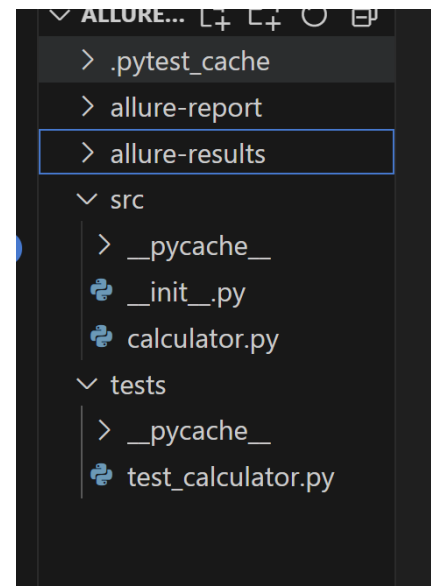
```

def test_addition(self, calc):
    with allure.step("مرحله ۱: جمع ۲ و ۳"):
        result = calc.add(2, 3)
        allure.attach(f"داده ورودی: a=2, b=3", name="داده ورودی",
attachment_type=AttachmentType.TEXT)
        with allure.step("مرحله ۲: بررسی نتیجه"):
            assert result == 5, "جمع ۲ و ۳ باید ۵ باشد"

    @allure.feature("ماشین حساب")
    @allure.severity(allure.severity_level.CRITICAL)
    @allure.description("تست عمدی شکست برای تقسیم بر صفر")
    def test_division_failure(self, calc):
        with allure.step("مرحله ۱: تلاش برای تقسیم ۱۰ بر ۰"):
            with pytest.raises(ValueError) as exc_info:
                calc.divide(10, 0)
            allure.attach(str(exc_info.value), name="پیام خطا",
attachment_type=AttachmentType.TEXT)
            with allure.step("مرحله ۲: تأیید استثنا"):
                assert str(exc_info.value) == "تقسیم بر صفر ممکن نیست..."

```

ساختار پروژه در vs code



بررسی امکان توسعه و مشارکت

با موفقیت در اجرای تست‌ها و مشاهده گزارش‌ها، پروژه Allure را که یک پروژه متن‌باز است بررسی کردم. ابتدا مستندات رسمی در GitHub را مطالعه کردم تا با فرآیند کانتریبیوت (Contribute) آشنا شوم. سپس کد منبع را از مخزن رسمی فورک کردم و با استفاده از IntelliJ و Gradle آن را ساختم. با رفع خطاها و اعمال تغییرات فرمت، توانستم بیلد را کامل کنم. اما هنوز نتوانستم آن را توسعه دهم.

3. گزارش کار با report portal :

<https://github.com/reportportal/reportportal>

نصب و راه‌اندازی پیش‌نیازها

برای استفاده از ReportPortal، ابتدا نیاز به نصب Docker Desktop داشتم که از وبسایت رسمی Docker نسخه مناسب ویندوز ۱۱ را دانلود کردم. پس از اجرای فایل نصب و انتخاب گزینه‌های پیش‌فرض مثل WSL ۲، برنامه را باز کردم و با دیدن آیکون نهنگ در نوار وظیفه، تأیید کردم که Docker فعال است. در تنظیمات Docker، حداقل ۲ CPU و ۶ گیگابایت RAM اختصاص دادم تا عملکرد بهتری داشته باشد. این مرحله پایه‌ای برای اجرای کانترینرهای ReportPortal بود و بدون مشکل پیش رفت.

دانلود و راه‌اندازی ReportPortal

برای راه‌اندازی ReportPortal، ابتدا با استفاده از دستور زیر

```
C:\Users\Mohadese\ReportPortal>curl -LO https://raw.githubusercontent.com/reportportal/reportportal/master/docker-compose.yml
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 18730 100 18730 0 0 17890 0 0:00:01 0:00:01 --:--:-- 17923
```

در Command Prompt، فایل تنظیمات Docker Compose را از مخزن رسمی GitHub دانلود کردم و در پوشه‌ای به نام C:\Users\Mohadese\ReportPortal قرار دادم. قبل از اجرا، فایل docker-compose.yml را با Notepad باز کردم و رمز عبور پیش‌فرض RP_INITIAL_ADMIN_PASSWORD را به MySecurePass123 تغییر دادم تا امنیت بیشتری داشته باشم. سپس با اجرای دستور docker compose -p reportportal up -d، کانترینرها را در حالت پس‌زمینه راه‌اندازی کردم:

```
C:\Users\Mohadese\ReportPortal>docker compose -p reportportal up -d
[+] Running 61/64
  ✓opensearch Pulled          946.6s
  ✓analyzer-train Pulled      756.1s
  ✓postgres Pulled           566.4s
  ✓api Pulled                 705.0s
  ✓gateway Pulled            437.6s
  ✓analyzer Pulled           756.0s
  ✓index Pulled              262.4s
  ✓uat Pulled                611.9s
  ✓rabbitmq Pulled          458.8s
  ✓jobs Pulled               580.9s
  ✓ui Pulled                 228.1s
  ✓migrations Pulled        263.5s
```

```
[+] Running 16/16
  ✓Network reportportal_reportportal Created 0.1s
  ✓Volume "reportportal_storage" Created 0.0s
  ✓Volume "reportportal_opensearch" Created 0.0s
  ✓Volume "reportportal_postgres" Created 0.0s
  ✓Container reportportal-gateway-1 Started 1.7s
  ✓Container reportportal-rabbitmq-1 Healthy 32.6s
  ✓Container reportportal-ui-1 Started 1.6s
  ✓Container postgres Healthy 11.7s
  ✓Container reportportal-opensearch-1 Started 1.5s
  ✓Container reportportal-analyzer-train-1 Started 32.7s
  ✓Container reportportal-analyzer-1 Started 32.8s
  ✓Container reportportal-migrations-1 Started 12.2s
  ✓Container reportportal-index-1 Started 1.6s
  ✓Container reportportal-jobs-1 Started 32.9s
  ✓Container reportportal-uat-1 Started 12.2s
  ✓Container reportportal-api-1 Started 32.9s
```

پس از چند دقیقه، با دستور `docker ps -a` وضعیت کانتینرها را چک کردم و خروجی زیر را دریافت کردم:

```
C:\Users\Mohadese\ReportPortal>docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
830da92d381e	reportportal/service-jobs:5.14.0	"/bin/sh -c 'exec ja..."	41 seconds ago	Up 9 seconds (health: starting)
7ce6d9c665d2	reportportal/service-auto-analyzer:5.14.1	"/venv/bin/uwsgi --h..."	41 seconds ago	Up 9 seconds (health: starting)
e870a19d70f8	reportportal/migrations:5.14.0	"/entrypoint.sh up"	41 seconds ago	Exited (0) 22 seconds ago
0f765d2257dc	reportportal/service-index:5.14.0	"/app"	41 seconds ago	Up 40 seconds (healthy)
6313b30b27e1	reportportal/service-authorization:5.14.3	"sh -c 'java \${JAVA_..."	41 seconds ago	Up 7 seconds (health: starting)
a4b3b5edc15c	reportportal/service-auto-analyzer:5.14.1	"/venv/bin/uwsgi --h..."	41 seconds ago	Up 9 seconds (health: starting)
efd4a4846114	reportportal/service-api:5.14.1	"sh -c 'java \${JAVA_..."	41 seconds ago	Up 9 seconds (health: starting)
f0dd011cbad6	reportportal/service-ui:5.14.2	"/docker-entrypoint..."	42 seconds ago	Up 41 seconds (healthy)
73784e33e5b8	traefik:v2.11.24	"/entrypoint.sh --pr..."	42 seconds ago	Up 40 seconds
9f9d35a8bacf	opensearchproject/opensearch:2.19.2	"/opensearch-docker..."	42 seconds ago	Up 41 seconds (healthy)
baa669816c08	bitnami/postgresql:16.6.0	"/opt/bitnami/script..."	42 seconds ago	Up 41 seconds (healthy)
2d41643fa56c	bitnami/rabbitmq:3.13.7-debian-12-r5	"/opt/bitnami/script..."	42 seconds ago	Up 41 seconds (healthy)

این خروجی نشان داد که کانتینرها با موفقیت فعال هستند و آماده استفاده‌اند.

ورود به داشبورد و تنظیمات اولیه

پس از راه اندازی، مرورگر را باز کردم و به آدرس <http://localhost:8080> رفتم. با نام کاربری default و رمز عبور 1q2w3e وارد شدم و سپس در بخش پروفایل، رمز را به MySecurePass123 که قبلاً تنظیم کرده بودم تغییر دادم. برای اتصال تست‌های پایتون به ReportPortal، پلاگین pytest-reportportal را با دستور `pip install pytest-reportportal` نصب کردم و با اجرای `pip show pytest-reportportal`، نسخه ۵.۵.۲ را تأیید شده دریافت کردم. این مرحله اتصال پایتون به داشبورد را ممکن ساخت.

تنظیم و اجرای تست‌های پایتون

برای تست، یک پوشه پروژه به نام C:\Users\Mohadese\ReportPortal\MyTests در VS Code ایجاد کردم و فایل `pytest.ini` را با تنظیمات زیر ساختم:

```
pytest.ini x test_sample.py
pytest.ini
1 [pytest]
2 rp_endpoint = http://localhost:8080
3 rp_project = default_personal
4 rp_api_key = sample-test_y76eXKvws3iNCJ7BrSeznSNgQ5VZl300bd1bwtBvvGzshB9HZ_la5damQmWML3b
5 rp_launch = MyFirstLaunch
6 rp_launch_attributes = 'PyTest' 'Smoke'
7 rp_launch_description = 'First test run with ReportPortal'
```

سپس فایل `test_sample.py` را با کد زیر نوشتم:

```
import pytest
import logging
from reportportal_client import RPLogger

@pytest.fixture(scope="session")
def rp_logger():
    logger = logging.getLogger(__name__)
    logger.setLevel(logging.INFO)
    logging.setLoggerClass(RPLogger)
    return logger

def test_example(rp_logger):
    rp_logger.info("Starting test example")
    assert 1 + 1 == 3
    rp_logger.info("Test failed")
```

با رفتن به دایرکتوری پروژه و اجرای دستور `pytest --reportportal` در Command Prompt ، تست اجرا شد و خروجی به این صورت بود:

```
C:\Users\Mohadese\ReportPortal\MyTests>pytest --reportportal
===== test session starts =====
platform win32 -- Python 3.12.2, pytest-8.4.1, pluggy-1.6.0
rootdir: C:\Users\Mohadese\ReportPortal\MyTests
configfile: pytest.ini
plugins: allure-pytest-2.14.3, anyio-4.9.0, reportportal-5.5.2
collected 1 item

test_sample.py F [100%]

===== FAILURES =====
_____ test_example _____

rp_logger = <RPLogger test_sample (INFO)>

    def test_example(rp_logger):
        rp_logger.info("Starting test example")
>       assert 1 + 1 == 3
E       assert (1 + 1) == 3

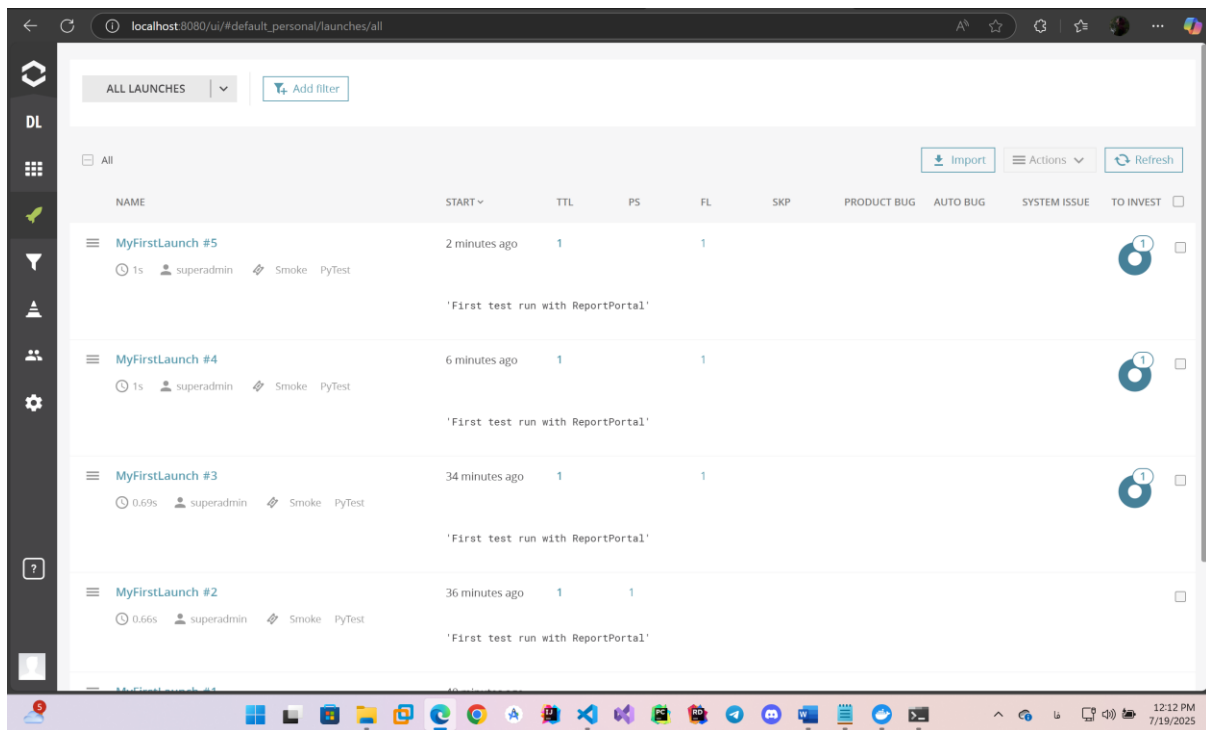
test_sample.py:14: AssertionError
----- Captured log call -----
INFO     test_sample:test_sample.py:13 Starting test example
===== short test summary info =====
FAILED test_sample.py::test_example - assert (1 + 1) == 3
===== 1 failed in 0.30s =====

C:\Users\Mohadese\ReportPortal\MyTests>
```

برای تست بیشتر، یک تست موفق با کد `assert 1 + 1 == 2` به فایل اضافه کردم و دوباره اجرا کردم که نتیجه آن در داشبورد به صورت یک تست موفق نمایش داده شد.

بررسی و تحلیل گزارش‌ها در داشبورد

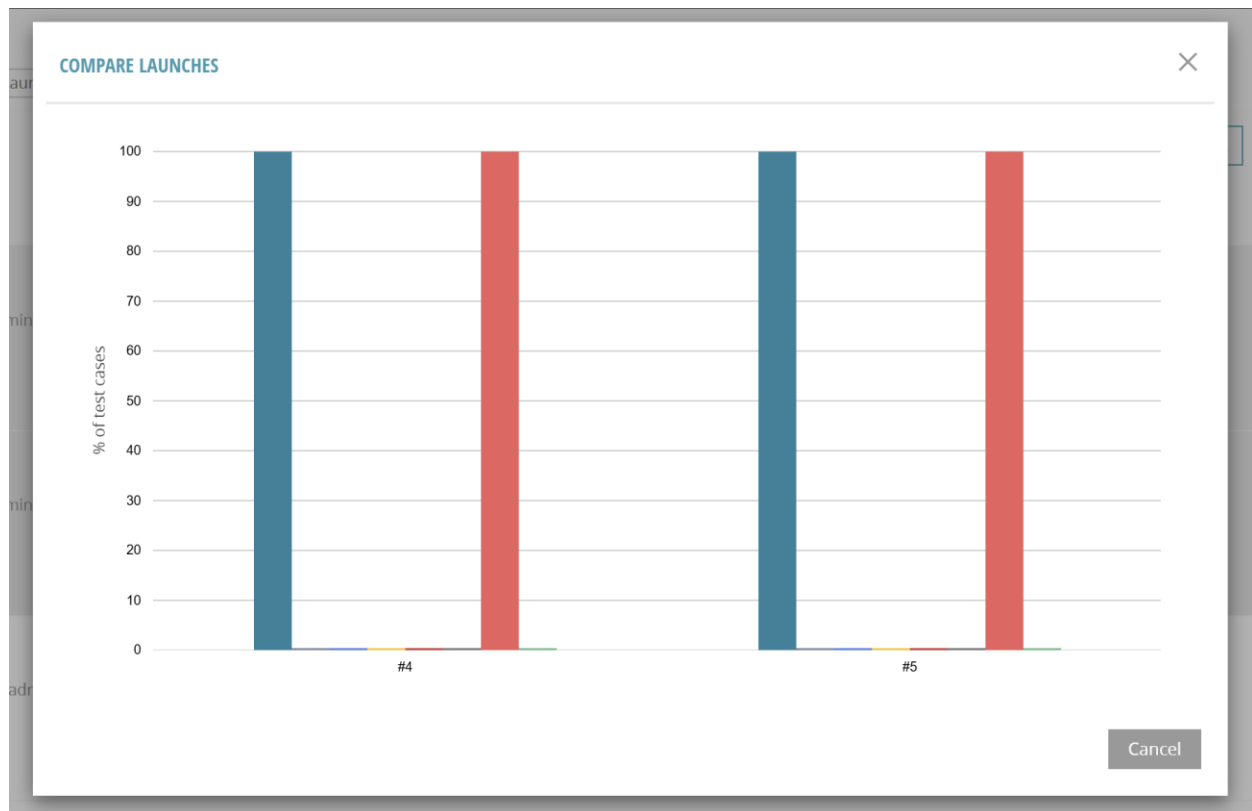
با ورود به <http://localhost:8080> و انتخاب لانچ `MyFirstLaunch`، داشبورد را بررسی کردم. بخش `Launches Table` لیستی از لانچ‌ها را نشان داد که شامل زمان اجرا و تعداد تست‌ها بود.



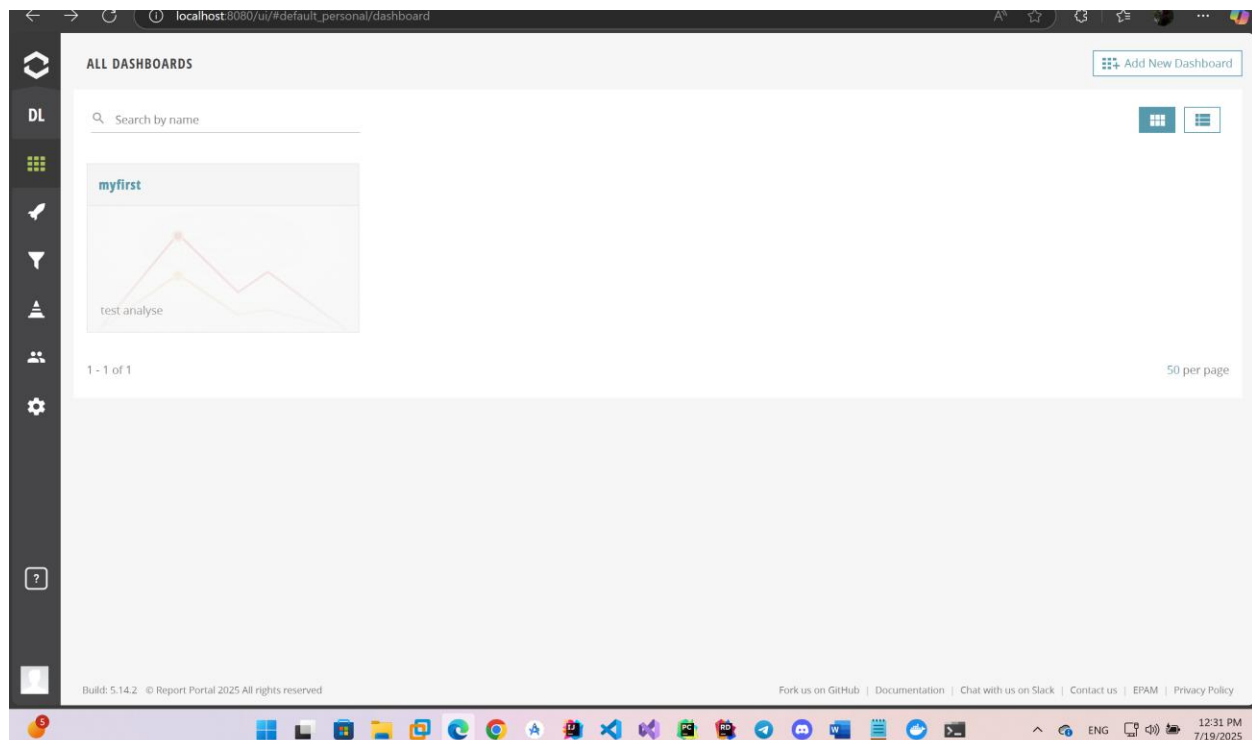
در صفحه launches میتوان به سری actionها بر روی تست ها اجرا کرد



مقایسه دو لانچ در بخش compare



ابتدا یک داشبورد برای خودم ایجاد کردم:



و این گزینه ها به عنوان ویجت هایی که میتوان به داشبورد اضافه کرد موجود هستند

1

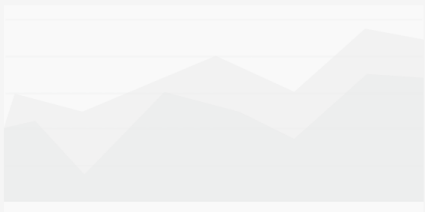
SELECT WIDGET TYPE

2

CONFIGURE WIDGET

3

SAVE



CHOOSE WIDGET TYPE FROM THE LIST BELOW

☐ Launch statistics chart

☐ Overall statistics

☐ Launches duration chart

☐ Launch execution and issue statistic

☐ Project activity panel

☐ Test-cases growth trend chart

☐ Investigated percentage of launches

☐ Launches table

☐ Unique bugs table

☐ Most failed test-cases table (TOP-50)

☐ Failed cases trend chart

☐ Non-passed test-cases trend chart

☐ Different launches comparison chart

☐ Passing rate per launch

☐ Passing rate summary

☐ Flaky test cases table (TOP-50)

☐ Cumulative trend chart

☐ Most popular pattern table (TOP-20)

☐ Component health check

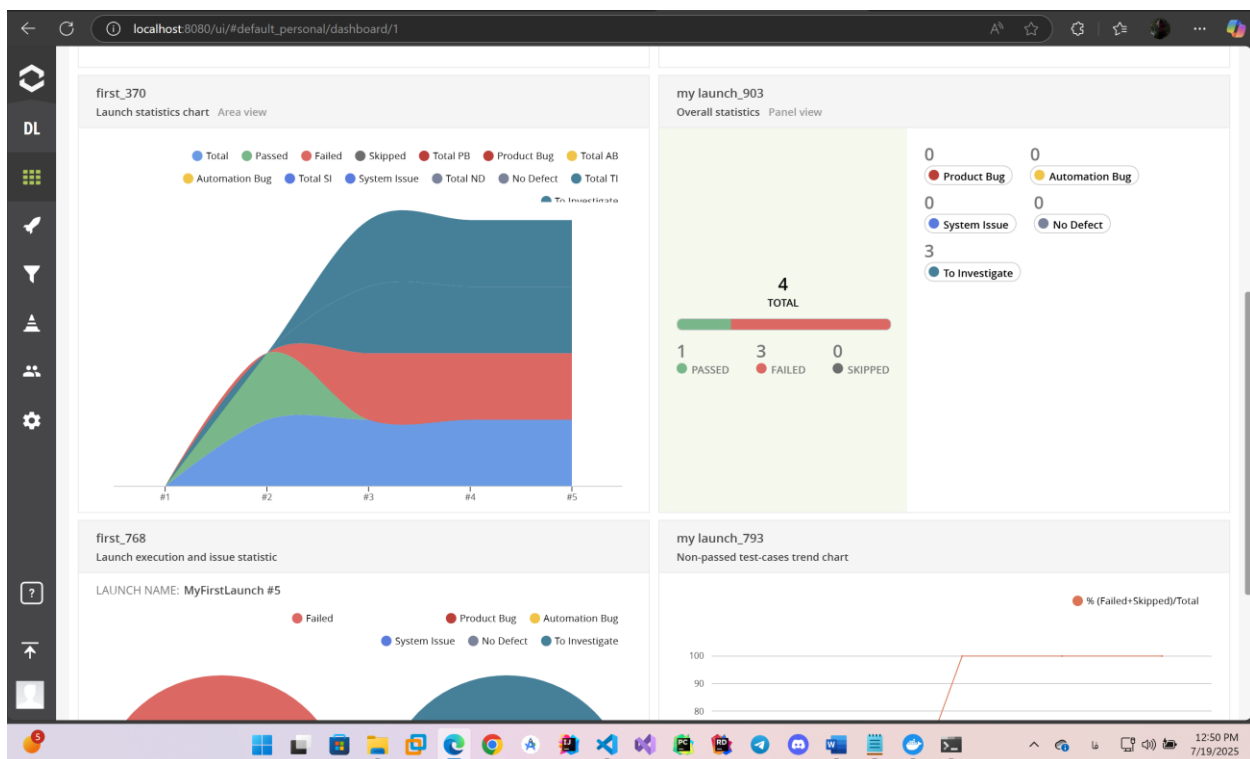
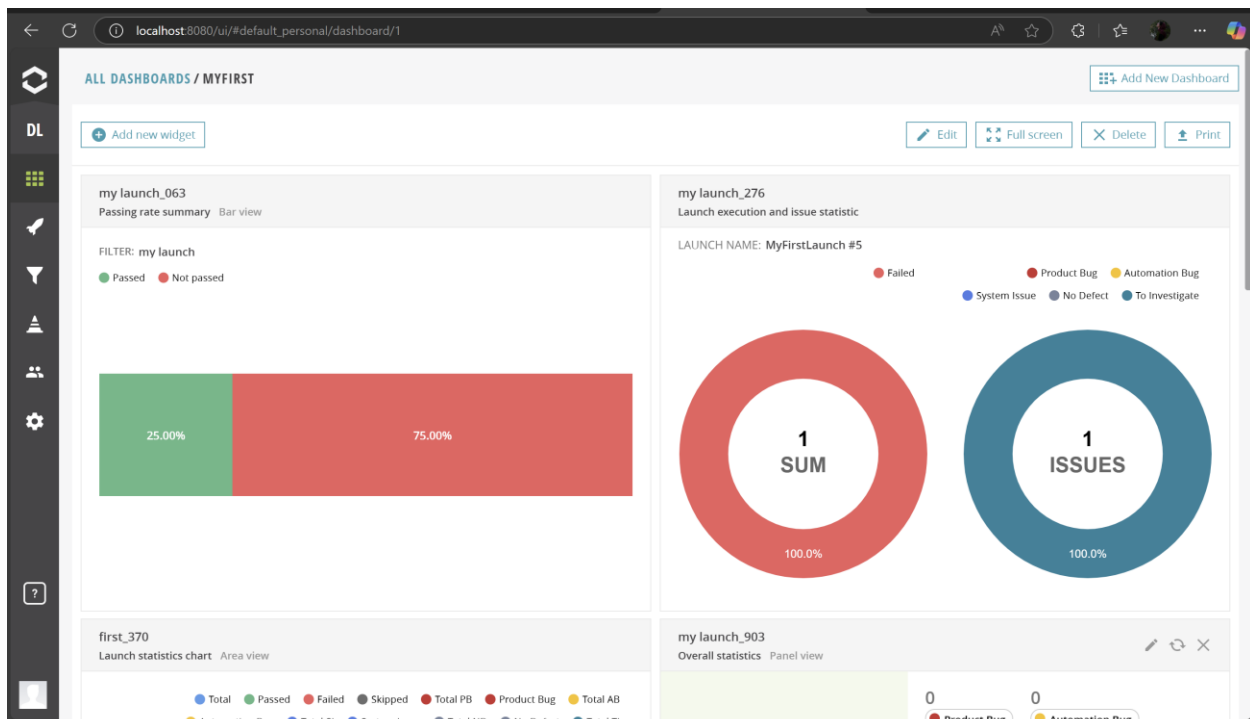
☐ Component health check (table view)

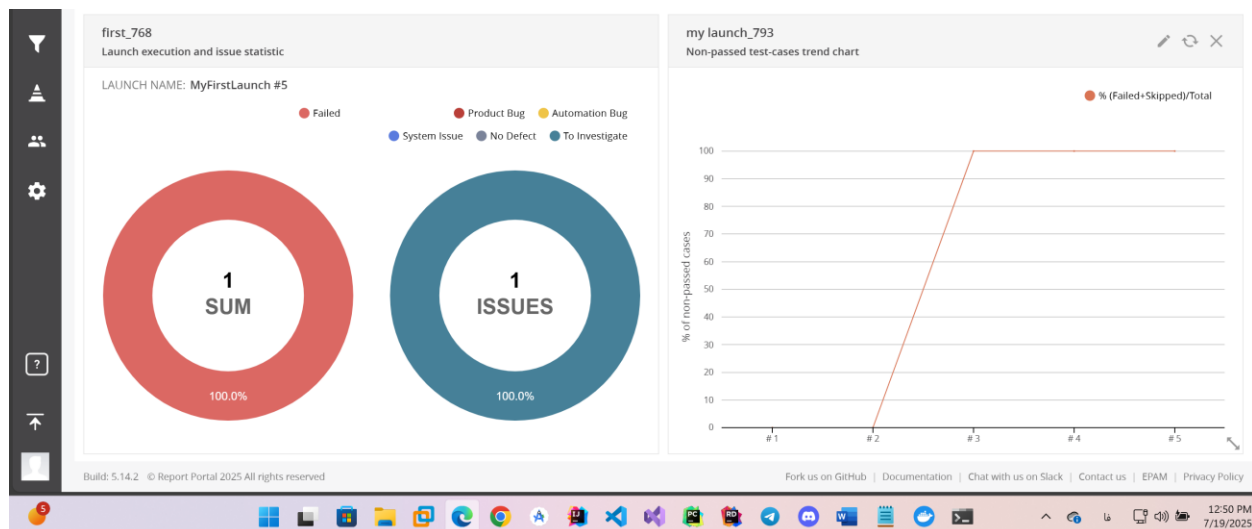
☐ Most time-consuming test cases widget (TOP-20)

☐ Test case search

Next step >

ویجت هایی که اضافه کردم و وصل کردم به لانچ های مربوط به دو تست:





نتیجه‌گیری و جمع‌بندی

این تجربه نشان داد که ReportPortal ابزاری قدرتمند برای مدیریت تست‌ها است و با امکانات متنوعی مثل تحلیل خودکار و فیلترهای پیشرفته، می‌تواند به بهبود فرآیند تست کمک کند. با اجرای `docker compose` `reportportal down`، کانتینرها را خاموش کردم تا منابع سیستم آزاد شوند. این پروژه پایه‌ای برای یادگیری بیشتر و شاید مشارکت در توسعه ReportPortal فراهم کرد و توانستم با فیچرهای اصلی آن آشنا شوم.

4. گزارش کار با ReportNG

مقدمه

هدفم پیاده‌سازی یک فرآیند تست خودکار با گزارش‌دهی پیشرفته بود. این گزارش با هدف مستندسازی مراحل نصب، پیکربندی، اجرا، و تحلیل گزارش‌های تولیدشده توسط ReportNG تهیه شده است تا به‌عنوان مرجعی تخصصی برای پروژه‌های آینده مورد استفاده قرار گیرد.

نصب و پیکربندی پیش‌نیازها

ابتدا با اجرای دستور `java -version` در Command Prompt، صحت نصب JDK تأیید شد. سپس در IntelliJ IDEA، یک پروژه Maven جدید با نام `ReportNGTest` در مسیر `C:\Users\Mohadese\IdeaProjects\ReportNGTest` ایجاد کردم. برای اطمینان از دسترسی به چارچوب TestNG و پلاگین ReportNG، وابستگی‌های لازم را در فایل `pom.xml` به‌شکل زیر تعریف کردم:

```
<dependencies>
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>6.14.3</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.uncommons</groupId>
    <artifactId>reportng</artifactId>
    <version>1.1.4</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.apache.velocity</groupId>
    <artifactId>velocity</artifactId>
    <version>1.7</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>com.google.inject</groupId>
    <artifactId>guice</artifactId>
    <version>4.2.3</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

پس از کلیک روی "Reload All Maven Projects" در IntelliJ، وابستگی‌ها با موفقیت دانلود شدند و محیط پروژه برای توسعه آماده شد.

کامند هایی که برای اعتبار سنجی بیشتر ران شدن:

```
PS C:\Users\Mohadese\IdeaProjects\ReportNGTest> mvn clean test "-DsuiteXmlFile=testng.xml" -e
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe:staticFieldBase has been called by com.google.inject.internal.aop.HiddenClassDefiner (file:/C:/Program%20Files/Apache/maven/apache-maven-3.9.1
0/lib/guice-5.1.0-classes.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.inject.internal.aop.HiddenClassDefiner
WARNING: sun.misc.Unsafe:staticFieldBase will be removed in a future release
[INFO] Error stacktraces are turned on.
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.example:ReportNGTest >-----
[INFO] Building ReportNGTest 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[WARNING] The POM for org.testng:testng:jar:5.14.3 is invalid, transitive dependencies (if any) will not be available: 1 problem was encountered while building the effective model for org.testng:testng:5.14.3
[ERROR] 'dependencies.dependency.systemPath' for org.testng:guice:jar must specify an absolute path but is ${basedir}/lib-supplied/guice-2.0.jar @
[WARNING] The POM for org.testng:testng:jar:5.14.4 is invalid, transitive dependencies (if any) will not be available: 1 problem was encountered while building the effective model for org.testng:testng:5.14.4
[ERROR] 'dependencies.dependency.systemPath' for org.testng:guice:jar must specify an absolute path but is /Users/cbeust/java/testng/target/checkout/lib-supplied/guice-2.0.jar @
```

```
2.0.jar @
[WARNING] The POM for com.beust:jcommander:jar:1.66 is missing, no dependency information available
[INFO] --- clean:3.2.0:clean (default-clean) @ ReportNGTest ---
[INFO] Deleting C:\Users\Mohadese\IdeaProjects\ReportNGTest\target
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.056 s
[INFO] Finished at: 2025-07-19T19:29:26+03:30
```

در ابتدا، اجرای این دستور با خطای "Cannot resolve class" مواجه شد که با اجرای mvn dependency:purge-local-repository و سپس "Reload All Maven Projects" در IntelliJ، مشکل ناسازگاری وابستگی‌ها برطرف شد.

در بخش src/test/java پروژه، یک پکیج با نام com.mohadese.reporting ایجاد کردم و فایل TestSample.java را با کد زیر توسعه دادم که شامل دو سناریوی تست بود:

```
package com.mohadese.reporting;
import org.testng.Assert;
import org.testng.Reporter;
import org.testng.annotations.Test;

public class TestSample {

    @Test
    public void testAdditionPass() {
        int a = 5;
        int b = 3;
        int expectedResult = 8;

        Reporter.log("Addition test started with numbers: " + a + " and " +
b);

        int actualResult = a + b;

        Assert.assertEquals(actualResult, expectedResult, "The sum of " + a +
" and " + b + " should be " + expectedResult);

        Reporter.log("Addition test completed successfully. Result: " +
actualResult);
    }

    @Test
    public void testAdditionFail() {
        int a = -5;
        int b = -3;
        int expectedResult = 8;

        Reporter.log("Negative addition test started with numbers: " + a + "
and " + b);

        int actualResult = a + b;

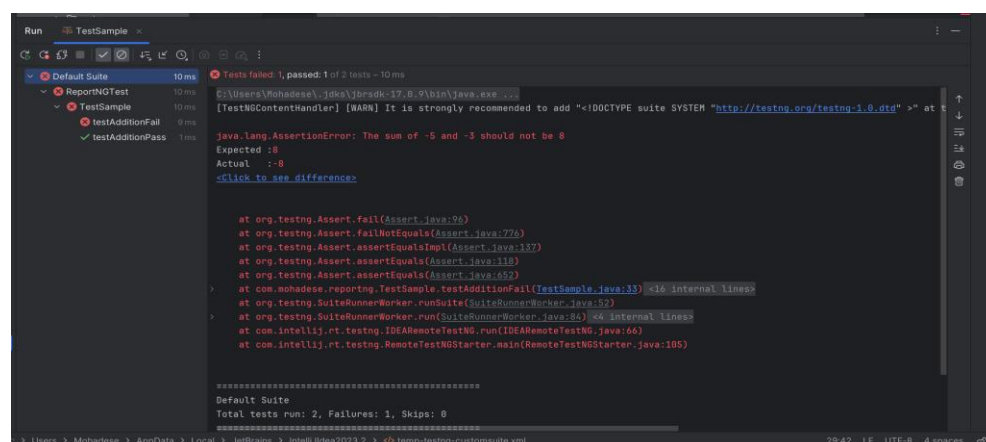
        Assert.assertEquals(actualResult, expectedResult, "The sum of " + a +
" and " + b + " should not be " + expectedResult);

        Reporter.log("Test completed");
    }
}
```

برای فعال سازی ReportNG به عنوان گزارش دهنده، فایل پیکربندی testng.xml را در ریشه پروژه با محتوای زیر تنظیم کردم:

```
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="ReportNG Suite">
  <test name="ReportNG Test">
    <classes>
      <class name="com.mohadese.reporting.TestSample"/>
    </classes>
  </test>
  <listeners>
    <listener class-name="org.uncommons.reporting.HTMLReporter"/>
    <listener class-name="org.uncommons.reporting.JUnitXMLReporter"/>
  </listeners>
</suite>
```

برای اجرای تست‌ها، ابتدا در IntelliJ فایل testng.xml را انتخاب نمودم و ران کردم. خروجی کنسول:



```
Run TestSample
Tests failed: 1, passed: 1 of 2 tests - 10 ms
[WARN] It is strongly recommended to add "<!DOCTYPE suite SYSTEM 'http://testng.org/testng-1.0.dtd'>" at the top of the testng.xml file.
java.lang.AssertionError: The sum of -5 and -3 should not be 8
Expected :8
Actual   :-8
Click to see difference.

at org.testng.Assert.fail(Assert.java:26)
at org.testng.Assert.failNotEquals(Assert.java:776)
at org.testng.Assert.assertEqualsImpl(Assert.java:137)
at org.testng.Assert.assertEquals(Assert.java:118)
at org.testng.Assert.assertEquals(Assert.java:652)
> at com.mohadese.reporting.TestSample.testAdditionFail(TestSample.java:35) ~16 Internal Lines
at org.testng.SuiteRunnerWorker.runSuite(SuiteRunnerWorker.java:52)
at org.testng.SuiteRunnerWorker.run(SuiteRunnerWorker.java:14) ~46 Internal Lines
at com.intellij.rt.testng.IDEARemoteTestNG.run(IDEARemoteTestNG.java:66)
at com.intellij.rt.testng.RemoteTestNGStarter.main(RemoteTestNGStarter.java:105)

*****
Default Suite
Total tests run: 2, Failures: 1, Skips: 0
*****
```

با این حال، در ابتدا پوشه test-output ایجاد نشد و با خطاهایی مثل "Class not found" مواجه شدم. با بررسی ساختار پکیج و اطمینان از تطابق نام کلاس در testng.xml با فایل جاوا، مشکل حل شد. پس از اجرا، پوشه test-output در ریشه پروژه ایجاد شد که شامل زیرپوشه html و فایل index.html در مسیر C:\Users\Mohadese\IdeaProjects\ReportNGTest\test-output\html\index.html بود. این فایل را با مرورگر Chrome باز کردم و گزارش رنگی شامل جدول نتایج مشاهده شد که تست testPass به رنگ سبز و تست testFail به رنگ قرمز نمایش داده شد. اسکرین‌شات این گزارش به شرح زیر است:

ReportNG - An HTML/XML rep...

دانشگاه ریمیکس آهنگ های ادر...

Test Results Report

localhost:63342/ReportNGTest/test-output/html/index.html?_ijt=-enfsg0gI9jk83vrrp9b0ctm36

GmailYouTubegolstanQueraFeed | LinkedIn(۲۲۵) ... صندوق ورودی ...

All Bookmarks

Test Results Report

Overview - Log Output

ReportNG Suite

ReportNG Test

Test Results Report

Log Output

Generated by TestNG with ReportNG at 20:04 BST on Saturday 19 July 2025
Mohadese@DESKTOP-72VPLHR / Java 17.0.9 (jetbrains.s.r.o.) / Windows 11 10.0 (amd64)

ReportNG Suite

	Duration	Passed	Skipped	Failed	Pass Rate
ReportNG Test	0.010s	1	0	1	50%
Total		1	0	1	50%

107°F
Sunny

Windows Taskbar Icons

8:05 PM
7/19/2025

ReportNG Test

Test duration: 0.010s

Failed Tests			
com.mohadese.reportng.TestSample			
testAdditionFail	0.009s	Negative addition test started with numbers: -5 and -3 java.lang.AssertionError: The sum of -5 and -3 should not be 8 expected [8] but found [-8]	
Passed Tests			
com.mohadese.reportng.TestSample			
testAdditionPass	0.001s	Addition test started with numbers: 5 and 3 Addition test completed successfully. Result: 8	

ReportNG Test

Test duration: 0.010s

Failed Tests

com.mohadesse.reportingng.TestSample		
testAdditionFail	0.009s	Negative addition test started with numbers: -5 and -3 java.lang.AssertionError: The sum of -5 and -3 should not be 8 expected [8] but found [-8] org.testing.Assert.fail(Assert.java:96) org.testing.Assert.failNotEquals(Assert.java:776) org.testing.Assert.assertEqualsImpl(Assert.java:137) org.testing.Assert.assertEquals(Assert.java:118) org.testing.Assert.assertEquals(Assert.java:652) com.mohadesse.reportingng.TestSample.testAdditionFail(TestSample.java:33) java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(Native Method) java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43) java.base/java.lang.reflect.Method.invoke(Method.java:568) org.testing.internal.MethodInvocationHelper.invokeMethod(MethodInvocationHelper.java:124) org.testing.internal.Invoker.invokeMethod(Invoker.java:583) org.testing.internal.Invoker.invokeTestMethod(Invoker.java:719) org.testing.internal.Invoker.invokeTestMethods(Invoker.java:989) org.testing.internal.TestMethodWorker.invokeTestMethods(TestMethodWorker.java:125) org.testing.internal.TestMethodWorker.run(TestMethodWorker.java:109) org.testing.TestRunner.privateRun(TestRunner.java:648) org.testing.TestRunner.run(TestRunner.java:505) org.testing.SuiteRunner.runTest(SuiteRunner.java:455) org.testing.SuiteRunner.runSequentially(SuiteRunner.java:450) org.testing.SuiteRunner.privateRun(SuiteRunner.java:415) org.testing.SuiteRunner.run(SuiteRunner.java:364) org.testing.SuiteRunnerWorker.runSuite(SuiteRunnerWorker.java:52) org.testing.SuiteRunnerWorker.run(SuiteRunnerWorker.java:84) org.testing.TestNG.runSuitesSequentially(TestNG.java:1208) org.testing.TestNG.runSuitesLocally(TestNG.java:1137) org.testing.TestNG.runSuites(TestNG.java:1049) org.testing.TestNG.run(TestNG.java:1017) com.intellij.rt.testing.IDEARemoteTestNG.run(IDEARemoteTestNG.java:66) com.intellij.rt.testing.RemoteTestNGStarter.main(RemoteTestNGStarter.java:105)

Passed Tests

com.mohadesse.reportingng.TestSample		
testAdditionPass	0.001s	Addition test started with numbers: 5 and 3 Addition test completed successfully. Result: 8

localhost:63342/ReportNGTest/test-output/html/index.html?_ijt=enfsg0gf9rk83vrrp9b0ctm36

Test Results Report
Overview · Log Output

ReportNG Suite
ReportNG Test

Log Output

Combined output from all calls to the log methods of the [TestNG Reporter](#).

Negative addition test started with numbers: -5 and -3
Addition test started with numbers: 5 and 3
Addition test completed successfully. Result: 8

برای سفارشی سازی میتوان در فایل CSS تغییراتی ایجاد کرد که جنبه بصری آن متفاوت شود.

نتیجه‌گیری و جمع‌بندی

تجربه کار با ReportNG نشان داد که این ابزار با ارائه گزارش‌های HTML رنگی و قابلیت سفارشی‌سازی، راهکاری کارآمد برای مدیریت تست‌های خودکار است. غلبه بر چالش‌های اولیه مثل بارگذاری وابستگی‌ها و پیکربندی صحیح، مهارت‌هایم را در عیب‌یابی و بهینه‌سازی فرآیند تست تقویت کرد. این پروژه منبعی ارزشمند برای ارائه در محیط‌های حرفه‌ای فراهم آورد و پایه‌ای برای توسعه بیشتر در حوزه تست خودکار ایجاد کرد.